

DataPact

Know When Your Data Breaks, and Why

Low-Fidelity Wireframes & Design Rationale

INFO 490 — Project 1, Assignment 1, Part 3 — Team 15
Ashok Chacko · Connor Slattery · Hriday Agarwal · Tejas Jaggi
University of Illinois Urbana-Champaign

AI AND DESIGN TOOL DISCLOSURE

The following tools were used during the design and production of these wireframes:

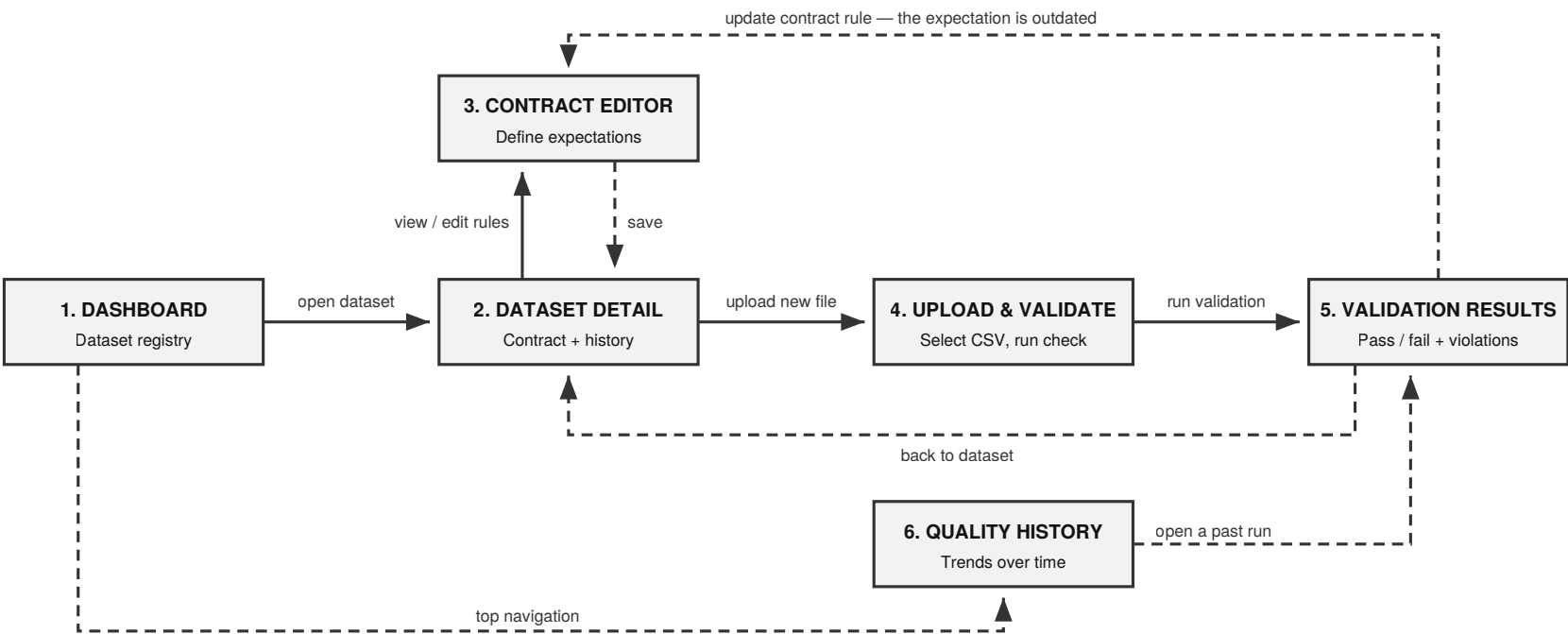
Tool	Version	How it was used
OpenAI ChatGPT	GPT-5.6 Luna	Brainstorming the screen set and critiquing draft layouts
Anthropic Claude	Opus 5	Reviewing the screen set, identifying missing screens and states, drafting design rationale, and generating the HTML/CSS used to render this document

These wireframes were produced by rendering team-specified HTML/CSS structures to PDF rather than by hand sketching. All screen decisions, layout choices, element placement, and design rationale were reviewed, revised, and approved by the team. No proprietary design software was used.

Accessed September 7, 2026.

SCREENFLOW

Primary user journey and return paths across the six screens



Legend

Solid arrow — primary forward path through the workflow

Dashed arrow — return, revision, or review path

Six screens in total; several are re-entered from more than one point.

Key loop: Validation Results → Contract Editor

A failed check does not always mean the incoming data is wrong. If the business has legitimately changed, the analyst updates the expectation instead of the file. This return path is what makes DataPact more than a one-time file checker.

SCREEN 1 — DASHBOARD / DATASET REGISTRY

Landing screen · low-fidelity

DATAPACT

Datasets · Upload · Quality History

My Datasets

+ Register Dataset

Search datasets...

Status: All ▼

Dataset	Source Team	Last Validated	Rules	Status
Monthly Enrollment Export	Registrar	2026-09-01 08:14	11	FAIL
Weekly Sales Extract	Finance	2026-09-04 06:02	14	PASS
Facilities Work Orders	Operations	2026-08-28 17:40	7	PASS
Sensor Readings — Lab 3	Research	2026-09-05 11:23	9	FAIL

Rows are clickable → Dataset Detail

EMPTY STATE (NO DATASETS REGISTERED)

No datasets registered yet.
Register your first dataset to begin validating recurring data.

+ Register Dataset

KEY UI ELEMENTS

- Persistent top navigation: Datasets, Upload, Quality History
- Page title and primary action button "+ Register Dataset"
- Text search field (filters by dataset name)
- Status filter dropdown: All / Passing / Failing
- Registry table: Dataset, Source Team, Last Validated, Rules, Status
- PASS / FAIL status badge in a single scannable column
- Empty-state message replacing the table when no datasets exist

INTERACTIONS

- Type in search to filter the table by name
- Select a status to narrow the list to failing datasets
- Click any row to open that dataset
- Click "+ Register Dataset" to open a short form (name, source team, description)

NAVIGATION

- **From:** application entry point
- **To:** Dataset Detail (row click) · Upload & Validate (nav) · Quality History (nav)

PAGE PURPOSE

Let the analyst see, at a glance, which recurring datasets exist and whether the most recent file for each one passed or failed, then open the dataset that needs attention.

SCREEN 2 — DATASET DETAIL

Single dataset overview · low-fidelity

DATAPACT

Datasets · Upload · Quality History

Datasets / Monthly Enrollment Export

Monthly Enrollment Export

Source: Registrar · Received monthly · CSV

Upload New File

CURRENT DATA CONTRACT

View / Edit Rules

Version 3 · 11 rules across 6 columns · Last edited 2026-08-19

LATEST VALIDATION

View Results

FAIL enrollment_2026_09.csv · 2026-09-01 08:14 · 2 of 11 rules failed · 51 rows affected

VALIDATION HISTORY

Date	File	Rules Failed	Rows Affected	Status
2026-09-01	enrollment_2026_09.csv	2	51	FAIL
2026-08-01	enrollment_2026_08.csv	0	0	PASS
2026-07-01	enrollment_2026_07.csv	1	8	FAIL

History rows are clickable → that run's Validation Results

KEY UI ELEMENTS

- Breadcrumb back to the registry
- Dataset identity block: name, source team, cadence, format
- Primary action "Upload New File"
- Panel 1 — Current Data Contract: version, rule count, last edited, "View / Edit Rules"
- Panel 2 — Latest Validation: status badge, file name, timestamp, failure counts, "View Results"
- Panel 3 — Validation History table of previous runs

INTERACTIONS

- Open the contract to inspect or change expectations
- Start a new validation for this dataset
- Open the latest run, or any earlier run from the history table
- Return to the registry via the breadcrumb

NAVIGATION

- **From:** Dashboard row click; return from Contract Editor or Validation Results
- **To:** Contract Editor · Upload & Validate · Validation Results

PAGE PURPOSE

Bring everything about one recurring dataset into a single view: what it is expected to contain, how the most recent file performed, and what has happened across previous files.

SCREEN 3 — CONTRACT EDITOR

Define the expectations for a dataset · low-fidelity

DATAPACT

Datasets · Upload · Quality History

Datasets / Monthly Enrollment Export / Contract

Data Contract — Rules for this dataset

Editing draft of Version 4 · based on Version 3

student_id

text

Not null

Unique

credit_hours

integer

Not null

Range 0–24

status

text

▲ collapse

Rule type: Allowed values

Values: ACTIVE, INACTIVE

Remove

Rule type: Not null

(no parameters required)

Remove

+ Add Rule

The parameter field changes with the selected rule type, so an invalid combination cannot be entered.

term_code

text

Format: ^[0-9]{4}[A-Z]\$

enrolled_on

date

Not null

+ Add Column

Cancel

Save Contract

KEY UI ELEMENTS

- Breadcrumb and draft-version indicator
- One collapsed row per column: column name, expected type, and its rules shown as short chips
- One expanded row showing the rule editor for the column being worked on
- Rule-type dropdown and a parameter field whose label adapts to the selected type
- "Remove", "+ Add Rule", "+ Add Column" controls
- "Save Contract" as the single primary action, bottom right

INTERACTIONS

- Expand a column to view or edit its rules; collapse it when finished
- Choose a rule type, then enter its parameters
- Add or remove rules and columns
- Save, which creates a new contract version rather than overwriting the old one

NAVIGATION

- **From:** Dataset Detail, or Validation Results with the failing rule pre-selected
- **To:** Dataset Detail on save or cancel

PAGE PURPOSE

Let the analyst write down, in one place, what a recurring dataset is supposed to contain, and revise those expectations when the underlying reality legitimately changes.

DataPact — Team 15 — Wireframes — p. 5

SCREEN 4 — UPLOAD & VALIDATE

Submit a new file for checking · low-fidelity

DATAFACT

Datasets · Upload · Quality History

Validate a File

Dataset: Monthly Enrollment Export ▼

Contract: Version 3 (current) ▼

Drop a CSV file here
or click to browse

enrollment_2026_09.csv · 412 KB · 4,812 rows · 6 columns detected

Remove

Validation runs when you upload a file. CSV format only.

Cancel

Run Validation

IN-PROGRESS STATE (AFTER "RUN VALIDATION")

Validating 4,812 rows against 11 rules...

KEY UI ELEMENTS

- Dataset selector, pre-filled when the user arrives from a dataset
- Contract version selector, defaulting to the current version
- Large dashed drop zone for the CSV file
- Selected-file summary: name, size, row count, column count
- Static scope note stating that validation runs on upload and CSV only
- "Run Validation" as the single primary action
- Separate in-progress state with a determinate progress indicator

INTERACTIONS

- Choose the dataset and the contract version to check against
- Drag a file onto the drop zone, or click to browse
- Remove the selected file and choose a different one
- Start validation, then watch progress until results appear

NAVIGATION

- **From:** Dataset Detail ("Upload New File") or the top navigation
- **To:** Validation Results when the run completes; Dataset Detail on cancel

PAGE PURPOSE

Let the analyst submit the newest version of a recurring dataset and check it against a chosen set of expectations, with clear feedback that the check is running.

DataPact — Team 15 — Wireframes — p. 6

SCREEN 5 — VALIDATION RESULTS

Outcome of one run, with violation detail expanded in place · low-fidelity

DATAPACT

Datasets · Upload · Quality History

Datasets / Monthly Enrollment Export / Run 2026-09-01

FAIL

enrollment_2026_09.csv
2026-09-01 08:14 · checked against Contract v3

Rules Evaluated	Rules Failed	Rows Affected	Total Rows
11	2	51	4,812

Column	Rule	Rows Affected	Sample Values
credit_hours	Range 0–24	47	-3, -1, -12

status · Allowed values

▲ collapse

4 rows in **status** contain the value **AUDIT**. The rule permits only ACTIVE and INACTIVE.

Row 118: AUDIT · Row 442: AUDIT · Row 2,910: AUDIT · Row 3,377: AUDIT

Flag Data IssueUpdate Contract Rule

A failure can mean the data is wrong, or that this expectation is out of date. The analyst decides which; the system does not assume.

If every rule passes, the violations table is replaced by: "All 11 rules passed. No violations found."

KEY UI ELEMENTS

- Result banner: large PASS/FAIL verdict, file name, timestamp, contract version used
- Summary strip: rules evaluated, rules failed, rows affected, total rows
- Violations table, one row per failed rule, each expandable in place
- Expanded violation: plain-language explanation, sample offending values with row numbers
- Two competing actions: "Flag Data Issue" and "Update Contract Rule"

INTERACTIONS

- Expand or collapse any violation without leaving the page
- Record that the incoming file is at fault ("Flag Data Issue")
- Open the contract with the failing rule pre-selected ("Update Contract Rule")

NAVIGATION

- **From:** completed run on Upload & Validate; history rows on Dataset Detail or Quality History
- **To:** Contract Editor (rule pre-selected) · Dataset Detail

PAGE PURPOSE

State plainly whether the file passed, show the specific rows and values behind each failure, and require the analyst to decide whether the data is wrong or the expectation is out of date.

SCREEN 6 — QUALITY HISTORY & TRENDS

Patterns across many runs · low-fidelity

DATAPACT

Datasets · Upload · Quality History

Quality History

Dataset: Monthly Enrollment Export ▾Last 6 runs ▾

Rules failed per validation run

Month	Rules Failed
Apr	2
May	1
Jun	0
Jul	1
Aug	0
Sep	2

Most frequent violations

Column	Rule	Times Failed	Last Occurrence
credit_hours	Range 0–24	3 of 6 runs	2026-09-01
status	Allowed values	1 of 6 runs	2026-09-01
student_id	Unique	1 of 6 runs	2026-07-01

Contract change history

v3 · 2026-08-19 — Tightened credit_hours range to 0–24

v2 · 2026-06-02 — Added uniqueness rule on student_id

v1 · 2026-04-11 — Initial contract

Violation rows are clickable → the runs where that rule failed

KEY UI ELEMENTS

- Dataset selector and run-range filter
- Bar chart of rules failed per run over time
- "Most frequent violations" table: column, rule, how often it failed, last occurrence
- Contract change history listing each version with a one-line reason

INTERACTIONS

- Switch dataset or change the range to refilter both the chart and the tables
- Click a frequent-violation row to see the runs in which it occurred

NAVIGATION

- From:** top navigation, or Dataset Detail
- To:** Validation Results for a specific run

DESIGN NOTE

- Contract changes appear beside the failure trend on purpose: a drop in failures is only meaningful if the reader can see whether the expectations changed at the same time.

PAGE PURPOSE

Move the analyst from a single file to the pattern across many, showing which problems recur, whether quality is improving, and how the expectations themselves have changed.

DESIGN RATIONALE

Why the screens are structured and arranged this way

Structure of the screen set

DataPact has six screens. We deliberately resisted adding more. Two decisions shaped the final set.

First, we **merged violation detail into Validation Results** rather than giving it a page of its own. Inspecting a single failure and seeing the list of failures are parts of one task, and separating them would force the analyst to lose the list in order to examine one item. Expanding a row in place keeps both in view.

Second, we **separated Quality History from Dataset Detail**. Dataset Detail answers "what happened to the last file"; Quality History answers "what keeps happening." These are different questions asked at different moments, and combining them produced a page that did neither well.

We also drew two states that are easy to omit at the wireframe stage but matter in use: the **empty state** on the Dashboard, for a user who has registered nothing yet, and the **in-progress state** on Upload, so the analyst is never left wondering whether the check is running.

Layout and hierarchy

Every screen carries the same top navigation in the same position, so the three main areas of the application are always reachable and always in the same place. Each screen has at most one filled primary button, placed top-right for creation actions and bottom-right for commit actions, so the next step is never ambiguous.

Information density increases as the user moves deeper. The Dashboard is a single scannable table. Dataset Detail is three grouped panels. The Contract Editor and Validation Results are the two dense screens, and both use expansion to keep only one item detailed at a time.

The central interaction decision

The most important design choice in the application appears on Validation Results. When a rule fails, the interface does not tell the analyst that the data is wrong. It presents two adjacent actions of equal visual weight: **Flag Data Issue** and **Update Contract Rule**.

This reflects the product's core idea. A dataset that historically contained only ACTIVE and INACTIVE may legitimately begin carrying AUDIT because the organization introduced a new status. The file breaks the existing rule, but the data is not incorrect. Presenting one action would encode an assumption the system is not entitled to make, so the interface requires a human judgment and records which was chosen.

This is also why the flow diagram includes a return path from Validation Results to the Contract Editor. Without it, DataPact would be a tool that checks files. With it, the expectations themselves are maintained over time.

Fidelity and scope

These wireframes are intentionally low-fidelity: plain boxes, grayscale, placeholder content, and no typographic or visual styling. The assignment evaluates clarity, reasoning, structure, and justification rather than polish, and committing to a visual design at this stage would imply decisions the team has not yet made.

Nothing outside the stated project scope is depicted. There are no authentication screens, alerts, scheduled ingestion, pipeline integrations, anomaly detection, or AI chat. Artificial intelligence appears nowhere in these wireframes, because in the initial application every validation decision is made by explicit rules.

DESIGN PRINCIPLES APPLIED

Each principle is listed only where it justified a specific decision

Principle	Where it applies	The decision it justified
Visibility of system status <i>Nielsen</i>	Screens 1, 4, 5	The Dashboard shows a PASS/FAIL badge for every dataset, so the analyst learns the health of all of them without opening any. Upload shows a determinate progress indicator during a run rather than an unexplained pause. Validation Results leads with the verdict as the largest element on the page, because it answers the only question the analyst arrived with.
Progressive disclosure <i>Cognitive Load Theory</i>	Screens 3, 5	These are the two dense screens. On the Contract Editor, collapsed rows show only what a rule says; the editing controls appear for the one column being worked on. On Validation Results, each failure expands in place. Cognitive load stays proportional to the current task instead of to the size of the contract or the number of failures.
Gestalt: proximity and similarity	Screens 1, 2, 6	Dataset Detail groups its content into three bordered panels answering three separate questions, so no section is read as part of another. On the Dashboard, status badges occupy a single vertical column of identical shapes, which the eye can scan without reading the surrounding text. Quality History places the failure chart directly beside the frequency table because they describe the same pattern at different resolutions.
Match between system and the real world <i>Nielsen</i>	Screens 3, 5	Failures are reported with the analyst's own data: actual offending values and their row numbers, not a count. The interface says "rules" where the analyst would say rules, and introduces the term "data contract" once, on the screen where those rules are gathered together.
Error prevention <i>Nielsen</i>	Screen 3	The rule parameter field changes with the selected rule type, so a numeric range cannot be entered where a list of allowed values is expected. Invalid combinations are made unavailable rather than validated after the fact.
Recognition rather than recall <i>Nielsen</i>	Screens 2, 5	Every validation run displays the contract version it was checked against, and Dataset Detail names the current version next to the latest result. The analyst never has to remember which expectations were in force when a given file was checked.
Fitts's Law	Screen 4	Upload has exactly two targets that matter, so the drop zone occupies a large share of the screen and the "Run Validation" button is large and consistently positioned, minimising the effort of the only two actions available.
Affordance <i>Norman</i>	Screen 4	The dashed border and the instruction inside the drop zone announce what can be done to that region, so its behaviour does not have to be learned or explained elsewhere.
Hick's Law	Screen 1	The status filter offers three options rather than a full set of facets, because the only question an analyst has on arrival is which datasets are failing. Additional filters would add decision time without adding value at that moment.
Consistency and standards <i>Nielsen</i>	All screens	The same navigation appears in the same position on every screen; the same status badge represents a result everywhere it appears; button wording is fixed across screens ("Run Validation", "Save Contract", "Upload New File") so the same action is never described two ways.
User control and freedom <i>Nielsen</i>	Screens 3, 5	Saving the contract creates a new version rather than overwriting the previous one, so a change to the expectations is never destructive and the history of what was expected remains intact.